

Tasks: webserver and Appserver

Task 1: Install nginx and run nginx on port number 81.

Objective:

Install and configure Nginx to run as a web server on port 81.

Purpose:

To serve web pages through Nginx using a custom HTTP port.

Implementation:

1. Install Nginx: `yum install nginx -y`
2. Edit Nginx configuration: `nano /etc/nginx/nginx.conf`
3. Change the listen port to: listen 81;
5. Start Nginx: `systemctl restart nginx`
6. Verify status: `systemctl status nginx`
7. Verify port: `lsof -i tcp:81`

```
[root@ip-172-31-28-114 conf]# lsof -i tcp:81
COMMAND  PID  USER  FD   TYPE DEVICE SIZE/OFF NODE NAME
nginx    31590 root    8u   IPv4 119461      0t0  TCP *:http (LISTEN)
nginx    31590 root    9u   IPv6 119462      0t0  TCP *:http (LISTEN)
nginx    31591 nginx   8u   IPv4 119461      0t0  TCP *:http (LISTEN)
nginx    31591 nginx   9u   IPv6 119462      0t0  TCP *:http (LISTEN)
nginx    31592 nginx   8u   IPv4 119461      0t0  TCP *:http (LISTEN)
nginx    31592 nginx   9u   IPv6 119462      0t0  TCP *:http (LISTEN)
[root@ip-172-31-28-114 conf]# ls
catalina catalina.properties jaspic-providers.xml logging.properties tomcat-users.xml web.xml
catalina.policy context.xml jaspic-providers.xsd server.xml tomcat-users.xsd
[root@ip-172-31-28-114 conf]# cd -
[root@ip-172-31-28-114 ~]# ls
[root@ip-172-31-28-114 ~]# cd /etc/nginx/
[root@ip-172-31-28-114 nginx]# ls
conf.d fastcgi.conf fastcgi_params koi-utf mime.types nginx.conf scgi_params uwsgi_params win-utf
default.d fastcgi.conf.default fastcgi_params.default koi-win mime.types.default nginx.conf.default scgi_params.default uwsgi_params.default
# For more information on configuration, see:
#   * Official English Documentation: http://nginx.org/en/docs/
#   * Official Russian Documentation: http://nginx.org/ru/docs/
user nginx;
```

```
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;

# Load dynamic modules. See /usr/share/doc/nginx/README.dynamic.
include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
}

http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile        on;
    tcp_nopush      on;
    keepalive_timeout 65;
    types_hash_max_size 4096;

    include         /etc/nginx/mime.types;
    default_type    application/octet-stream;

    # Load modular configuration files from the /etc/nginx/conf.d directory.
    # See http://nginx.org/en/docs/nginx_core_module.html#include
    # for more information.
    include /etc/nginx/conf.d/*.conf;

    server {
        listen      80;
        listen      [::]:80;
        server_name _;
        root        /usr/share/nginx/html;

        # Load configuration files for the default server block.
        include /etc/nginx/default.d/*.conf;

        error_page 404 /404.html;
        location = /404.html {
            }
    }
}
```

```

}
[root@ip-172-31-28-114 nginx]# nano nginx.conf
[root@ip-172-31-28-114 nginx]# cat nginx.conf
# For more information on configuration, see:
# * Official English Documentation: http://nginx.org/en/docs/
# * Official Russian Documentation: http://nginx.org/ru/docs/

user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;

# Load dynamic modules. See /usr/share/doc/nginx/README.dynamic.
include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
}

http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

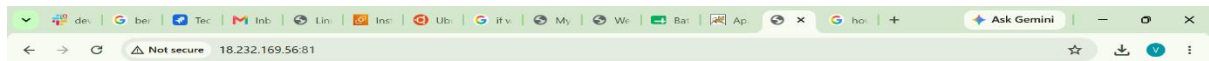
    sendfile        on;
    tcp_nopush      on;
    keepalive_timeout 65;
    types_hash_max_size 4096;

    include          /etc/nginx/mime.types;
    default_type     application/octet-stream;

    # Load modular configuration files from the /etc/nginx/conf.d directory.
    # See http://nginx.org/en/docs/nginx_core_module.html#include
    # for more information.
    include /etc/nginx/conf.d/*.conf;

    server {
        listen       81;
        listen       [::]:80;
        server_name  _;
        root          /usr/share/nginx/html;
    }
}

```



Welcome to techie horizon

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to fs.com/nginx.

Thank you for using nginx.



Conclusion:

Successfully installed and configured Nginx to run on port 81.

Task 2: Deploy a sample index.html file on nginx.

Objective:

Deploy a sample index.html file on the Nginx web server.

Purpose:

To host and display a web page using Nginx.

Implementation:

1. Create the HTML file: `nano /usr/share/nginx/html/index.html`
2. Add sample HTML content.
3. Save the file and exit.

4. Restart Nginx: systemctl restart nginx

5. Verify Nginx status: systemctl status nginx

6. Access the page in the browser: http://<server-public-ip-of-ec2>:81

```
[root@ip-172-31-28-114 ~]# cd /usr/share/nginx/html
[root@ip-172-31-28-114 html]# ls
404.html  50x.html  icons  index.html  nginx-logo.png  poweredby.png
[root@ip-172-31-28-114 html]# cat index.html
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to techie horizon</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

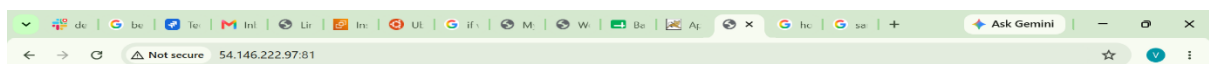
<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[root@ip-172-31-28-114 html]# |
```

```
[root@ip-172-31-28-114 html]# nano index.html
[root@ip-172-31-28-114 html]# cat index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample Web Page</title>
</head>
<body>

  <header>
    <h1>Welcome to My Sample Page 2</h1>
    <p>This is a basic HTML5 boilerplate template.</p>
  </header>

  <main>
    <section>
      <h2>About This Template</h2>
      <p>HTML elements use tags to structure web content. This section represents the main content area.</p>
      <p>Here is an example of an external link to
      <a href="https://www.w3schools.com" target="_blank">w3schools</a>.</p>
    </section>
    <section>
      <h2>Common HTML Elements</h2>
      <p>Web developers frequently use unordered lists for navigation or groupings:</p>
      <ul>
        <li><strong>Bold Text:</strong> Made using the strong tag.</li>
        <li><em>Italic Text:</em> Made using the em tag.</li>
        <li><u>Underlined Text:</u> Made using the u tag.</li>
      </ul>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Sample Company. All rights reserved.</p>
  </Footer>
</body>
</html>
[root@ip-172-31-28-114 html]#
```



Welcome to My Sample Page 2

This is a basic HTML5 boilerplate template.

About This Template

HTML elements use tags to structure web content. This section represents the main content area.

Here is an example of an external link to [W3Schools](https://www.w3schools.com).

Common HTML Elements

Web developers frequently use unordered lists for navigation or groupings:

- **Bold Text:** Made using the strong tag.
- *Italic Text:* Made using the em tag.
- Underlined Text: Made using the u tag.

© 2026 Sample Company. All rights reserved.



Conclusion:

Successfully deployed and verified the sample index.html page on Nginx running on port 81.

Task 3: Install Apache and run Apache on port number 82.

Objective:

Install and configure Apache HTTP Server to run on port 82..

Purpose:

To host and serve web pages using Apache on a custom HTTP port.

Implementation:

1. Install Apache: `yum install httpd -y`
2. Edit Apache configuration: `nano /etc/httpd/conf/httpd.conf`
3. Change the listening port: `Listen 82`
4. Start Apache: `systemctl start httpd`
5. Verify status: `systemctl status httpd`
6. Verify port: `lsof -i tcp:82`.

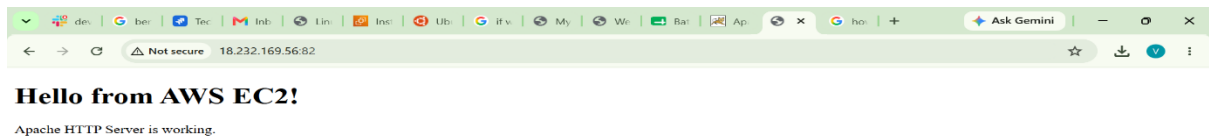
```
[root@ip-172-31-28-114 conf]# nano httpd.conf
[root@ip-172-31-28-114 conf]# cat httpd.conf
#
# This is the main Apache HTTP server configuration file. It contains the
# configuration directives that give the server its instructions.
# See <URL:http://httpd.apache.org/docs/2.4/> for detailed information.
# In particular, see
# <URL:http://httpd.apache.org/docs/2.4/mod/directives.html>
# for a discussion of each configuration directive.
#
# See the httpd.conf(5) man page for more information on this configuration,
# and httpd.service(8) on using and configuring the httpd service.
#
# Do NOT simply read the instructions in here without understanding
```

```
root@ip-172-31-28-114/etc/httpd/conf
# consult the online docs. You have been warned.
#
# Configuration and logfile names: If the filenames you specify for many
# of the server's control files begin with "/" (or "drive:/" for win32), the
# server will use that explicit path. If the filenames do "not" begin
# with "/", the value of ServerRoot is prepended -- so "/log/access_log"
# with ServerRoot set to "/www" will be interpreted by the
# server as "/www/log/access_log", where as "/log/access_log" will be
# interpreted as "/log/access_log".
#
# ServerRoot: The top of the directory tree under which the server's
# configuration, error, and log files are kept.
#
# Do not add a slash at the end of the directory path. If you point
# ServerRoot at a non-local disk, be sure to specify a local disk on the
# Mutex directive, if file-based mutexes are used. If you wish to share the
# same ServerRoot for multiple httpd daemons, you will need to change at
# least PidFile.
ServerRoot "/etc/httpd"
#
# Listen: Allows you to bind Apache to specific IP addresses and/or
# ports, instead of the default. See also the <VirtualHost>
# directive.
#
# Change this to Listen on a specific IP address, but note that if
# httpd.service is enabled to run at boot time, the address may not be
# available when the service starts. See the httpd.service(8) man
# page for more information.
#Listen 12.34.56.78:80
Listen 82
#
# Dynamic Shared Object (DSO) Support
#
# To be able to use the functionality of a module which was built as a DSO you
# have to place corresponding 'LoadModule' lines at this location so the
# directives contained in it are actually available before they are used.
# Statically compiled modules (those listed by 'httpd -l') do not need
# to be loaded here.
#
# Example:
# LoadModule foo_module modules/mod_foo.so
include conf.modules.d/*.conf
```

```

failed to restart apache.service: Unit apache.service not found.
[root@ip-172-31-28-114 conf]# systemctl restart httpd
[root@ip-172-31-28-114 conf]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset:
   Active: active (running) since Wed 2026-08-12 11:01:43 UTC; 20s ago
     Docs: man:httpd.service(8)
   Main PID: 41445 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Byt
     Tasks: 177 (limit: 1059)
    Memory: 13.9M
       CPU: 76ms
    CGroup: /system.slice/httpd.service
            └─41445 /usr/sbin/httpd -DFOREGROUND
              └─41446 /usr/sbin/httpd -DFOREGROUND
                └─41447 /usr/sbin/httpd -DFOREGROUND
                  └─41448 /usr/sbin/httpd -DFOREGROUND

```



Conclusion:

Successfully installed and configured Apache to run on port 82.

Task 4: Deploy a sample index.html file on Apache.

Objective:

Deploy a sample index.html file on the Apache web server.

Purpose:

To host and display a web page using Apache on port 82.

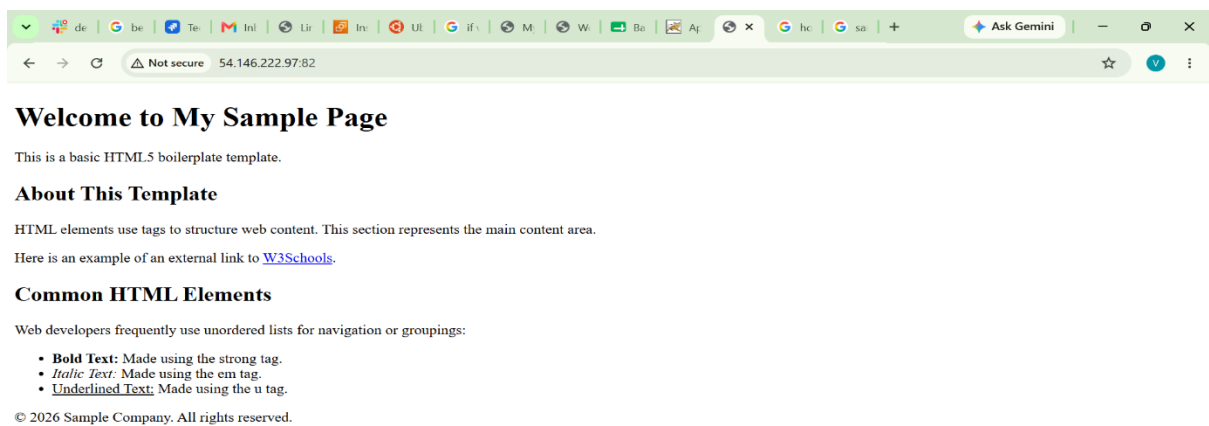
Implementation:

1. Create the HTML file: `nano /var/www/html/index.html`
2. Add sample HTML content and save the file.
3. Restart Apache: `systemctl restart httpd`
4. Verify Apache status: `systemctl status httpd`
5. Access the page in the browser: `http://<server-publicip-of-ec2>:82.`

```
[root@ip-172-31-28-114 ~]# cd /var/www/html
[root@ip-172-31-28-114 html]# ls
index.html
[root@ip-172-31-28-114 html]# cat index.html
<!DOCTYPE html>
<html>
<head>
  <title>My EC2 Server</title>
</head>
<body>
  <h1>Hello from AWS EC2!</h1>
  <p>Apache HTTP Server is working.</p>
</body>
</html>
[root@ip-172-31-28-114 html]# nano index.html
[root@ip-172-31-28-114 html]# cat index.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample Web Page</title>
</head>
<body>

  <headers>
    <h1>Welcome to My Sample Page</h1>
    <p>This is a basic HTML5 boilerplate template.</p>
  </headers>

  <main>
    <section>
      <h2>About This Template</h2>
      <p>HTML elements use tags to structure web content. This section represents the main content area.</p>
      <p>Here is an example of an external link to
        <a href="https://www.w3schools.com" target="_blank">W3Schools</a>.
      </p>
    </section>
  </main>
</body>
</html>
```



Conclusion:

Successfully deployed and verified the sample index.html page on Apache running on port 82.

Task 5: Install Apache tomcat on port number 8082.

Objective:

Install and configure Apache Tomcat to run on port 8082.

Purpose:

To deploy and run Java web applications using Apache Tomcat on a custom port.

Implementation:

1. Install Java: `yum install java`.
2. Download and extract Apache Tomcat : `wget <url- of-tomcat-tar-file>`.
3. Edit Tomcat configuration: `nano /opt/tomcat/conf/server.xml`
4. Change the connector port to 8082: `<Connector port="8082" ... />`
5. Start Tomcat: `/opt/tomcat/bin/startup.sh`

6. Verify the port: `lsf -i tcp :8082`

7. Access Tomcat in the browser: `http://<server-public-ip-ec2>:8082`.

```
[root@ip-172-31-28-114 ~]# cd /opt/
[root@ip-172-31-28-114 opt]# ls
aws
[root@ip-172-31-28-114 opt]# wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.57/bin/apache-tomcat-10.1.57.tar.gz
--2026-08-12 10:17:20-- https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.57/bin/apache-tomcat-10.1.57.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 14400273 (14M) [application/x-gzip]
Saving to: 'apache-tomcat-10.1.57.tar.gz'

apache-tomcat-10.1.57.tar.gz      100%[=====]
2026-08-12 10:17:20 (301 MB/s) - 'apache-tomcat-10.1.57.tar.gz' saved [14400273/14400273]

[root@ip-172-31-28-114 opt]# ls
apache-tomcat-10.1.57.tar.gz  aws
[root@ip-172-31-28-114 opt]# tar xvf apache-tomcat-10.1.57.tar.gz
apache-tomcat-10.1.57/conf/
apache-tomcat-10.1.57/conf/catalina.policy
apache-tomcat-10.1.57/conf/catalina.properties
apache-tomcat-10.1.57/conf/context.xml
apache-tomcat-10.1.57/conf/jaspic-providers.xml
apache-tomcat-10.1.57/conf/jaspic-providers.xsd
apache-tomcat-10.1.57/conf/logging.properties
apache-tomcat-10.1.57/conf/server.xml
```

```
apache-tomcat-10.1.57/bin/tool-wrapper.sh
apache-tomcat-10.1.57/bin/version.sh
[root@ip-172-31-28-114 opt]# ls
apache-tomcat-10.1.57  apache-tomcat-10.1.57.tar.gz  aws
[root@ip-172-31-28-114 opt]# cd apache-tomcat-10.1.57/
[root@ip-172-31-28-114 apache-tomcat-10.1.57]# ls
BUILDING.txt  CONTRIBUTING.md  LICENSE  NOTICE  README.md  RELEASE-NOTES  RUNNING.txt  bin  conf  lib  logs  temp  webapps  work
[root@ip-172-31-28-114 apache-tomcat-10.1.57]# cd bin/
[root@ip-172-31-28-114 bin]# ls
bootstrap.jar  catalina-tasks.xml  catalina.bat  catalina.sh  commons-daemon-native.tar.gz  commons-daemon.jar  configtest.bat  configtest.sh  daemon.sh  digest.bat  digest.sh  migrate.bat  migrate.sh  makebase.bat  makebase.sh  setclasspath.bat  setclasspath.sh  shutdown.bat  shutdown.sh  startup.bat  startup.sh  tomcat-native.tar.gz  tomcat-juli.jar  tool-wrapper.bat  tool-wrapper.sh  version.bat  version.sh
[root@ip-172-31-28-114 bin]# ./startup.sh
Using CATALINA_BASE:   /opt/apache-tomcat-10.1.57
Using CATALINA_HOME:   /opt/apache-tomcat-10.1.57
Using CATALINA_TMPDIR: /opt/apache-tomcat-10.1.57/temp
Using JRE_HOME:        /usr
Using CLASSPATH:        /opt/apache-tomcat-10.1.57/bin/bootstrap.jar:/opt/apache-tomcat-10.1.57/bin/tomcat-juli.jar
Tomcat started.
[root@ip-172-31-28-114 bin]# cd ..
[root@ip-172-31-28-114 apache-tomcat-10.1.57]# ls
BUILDING.txt  CONTRIBUTING.md  LICENSE  NOTICE  README.md  RELEASE-NOTES  RUNNING.txt  bin  conf  lib  logs  temp  webapps  work
[root@ip-172-31-28-114 apache-tomcat-10.1.57]# cd webapps
[root@ip-172-31-28-114 webapps]# ls
ROOT  docs  examples  host-manager  manager
[root@ip-172-31-28-114 webapps]# wget https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
--2026-08-12 10:20:40-- https://tomcat.apache.org/tomcat-7.0-doc/appdev/sample/sample.war
Resolving tomcat.apache.org (tomcat.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to tomcat.apache.org (tomcat.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4606 (4.5K)
Saving to: 'sample.war'

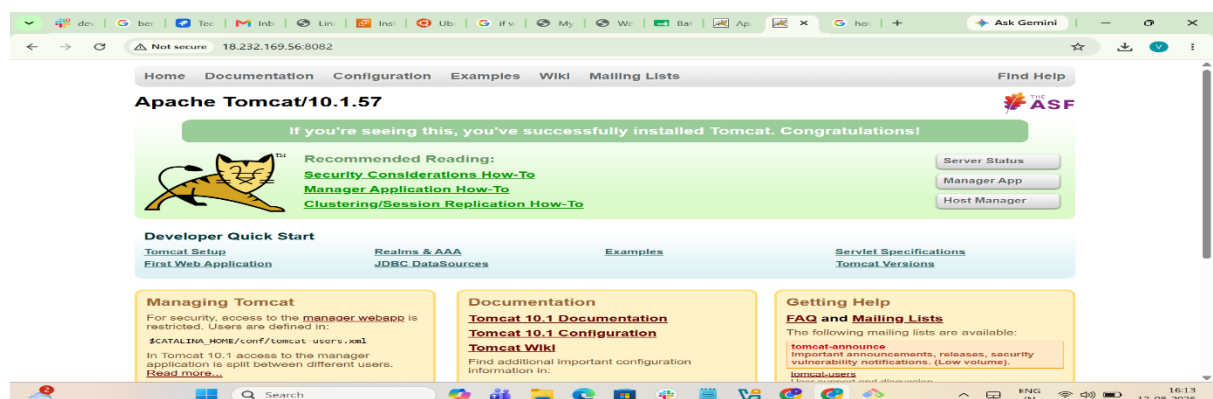
sample.war      100%[=====]

```

```
[root@ip-172-31-28-114 bin]# xc
[root@ip-172-31-28-114 bin]# sudo nano /etc/systemd/system/tomcat.service
[root@ip-172-31-28-114 bin]# sudo systemctl daemon-reload
[root@ip-172-31-28-114 bin]# sudo systemctl enable tomcat
Created symlink /etc/systemd/system/multi-user.target.wants/tomcat.service → /etc/systemd/system/tomcat.service.
[root@ip-172-31-28-114 bin]# sudo systemctl start tomcat
```

```
[root@ip-172-31-28-114 opt]# cd /opt/tomcat/bin/
[root@ip-172-31-28-114 bin]# sudo systemctl start tomcat
[root@ip-172-31-28-114 bin]# sudo systemctl status tomcat
● tomcat.service - Apache Tomcat
   Loaded: loaded (/etc/systemd/system/tomcat.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-08-12 10:35:48 UTC; 15s ago
   Process: 39782 ExecStart=/opt/tomcat/bin/startup.sh (code=exited, status=0/SUCCESS)
   Main PID: 39789 (java)
   Tasks: 29 (limit: 1059)
   Memory: 70.9M
   CPU: 4.516s
   CGroup: /system.slice/tomcat.service
           └─39789 /usr/lib/jvm/java-26-amazon-corretto.x86_64/bin/java -Djava.util.logging.config.file=/opt/tomcat/conf/logging.properties -

Aug 12 10:35:48 ip-172-31-28-114.ec2.internal systemd[1]: Starting tomcat.service - Apache Tomcat...
Aug 12 10:35:48 ip-172-31-28-114.ec2.internal startup.sh[39782]: Tomcat started.
Aug 12 10:35:48 ip-172-31-28-114.ec2.internal systemd[1]: Started tomcat.service - Apache Tomcat.
lines 1-14/14 (END)
```



Conclusion:

Successfully installed and configured Apache Tomcat to run on port 8082.

Task 6: Deploy a sample app on webapps.

Objective:

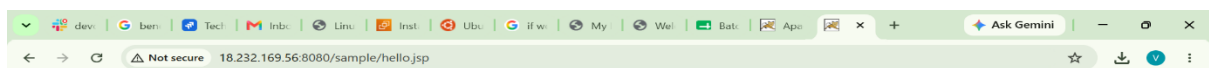
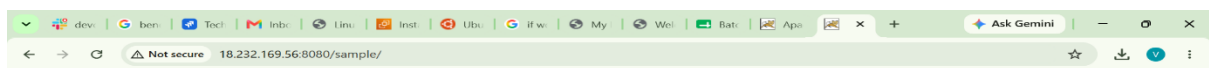
Deploy a sample web application in the Apache Tomcat webapps directory.

Purpose:

To deploy and run a web application using the Tomcat application server.

Implementation:

1. Download sample application directory in webapps folder : `wget <url- tar-file>`
2. Restart Tomcat: `systemctl restart tomcat`
3. Verify the application in the browser: `http://<server-public-ip-ec2>:8082/sample.`



Conclusion:

Successfully deployed and verified a sample application in the Tomcat webapps directory.

Task 7: Create a tomcat.service file for tomcat.

Objective:

Create and configure a tomcat.service file to manage Apache Tomcat using systemctl.

Purpose:

To manage Tomcat as a Linux system service for easy start, stop, restart, and automatic startup

Implementation:

1. Create the service file: `nano /etc/systemd/system/tomcat.service`
2. Configure JAVA_HOME, CATALINA_HOME, and Tomcat startup/shutdown commands.
3. Reload systemd: `systemctl daemon-reload`
4. Enable Tomcat: `systemctl enable tomcat`
5. Start Tomcat: `systemctl start tomcat`
6. Verify status: `systemctl status tomcat`

```
root@ip-172-31-28-114 bin]# nano /etc/systemd/system/tomcat.service
root@ip-172-31-28-114 bin]# sudo systemctl daemon-reload
root@ip-172-31-28-114 bin]# sudo systemctl enable tomcat
Created symlink /etc/systemd/system/multi-user.target.wants/tomcat.service → /etc/systemd/system/tomcat.service.
root@ip-172-31-28-114 bin]# sudo systemctl start tomcat

root@ip-172-31-28-114 bin]# sudo systemctl status tomcat
root@ip-172-31-28-114 bin]# sudo systemctl status tomcat
● tomcat.service - Apache Tomcat
   Loaded: loaded (/etc/systemd/system/tomcat.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-08-12 10:35:48 UTC; 15s ago
     Process: 39782 ExecStart=/opt/tomcat/bin/startup.sh (code=exited, status=0/SUCCESS)
    Main PID: 39789 (java)
       Tasks: 29 (limit: 1059)
      Memory: 70.9M
         CPU: 4.516s
    CGroup: /system.slice/tomcat.service
            └─39789 /usr/lib/jvm/java-26-amazon-corretto.x86_64/bin/java -Djava.util.logging.config.file=/opt/tomcat/conf/logging.properties -

Aug 12 10:35:48 ip-172-31-28-114.ec2.internal systemd[1]: Starting tomcat.service - Apache Tomcat...
Aug 12 10:35:48 ip-172-31-28-114.ec2.internal startup.sh[39782]: Tomcat started.
Aug 12 10:35:48 ip-172-31-28-114.ec2.internal systemd[1]: Started tomcat.service - Apache Tomcat.
lines 1-14/14 (END)
```

Conclusion:

Successfully created and configured the tomcat.service file and managed Tomcat using systemctl.

Task 8: Configure HA Proxy server

Objective:

Configure HAProxy as a load balancer using AWS EC2 instances.

Purpose:

To distribute incoming web traffic between multiple backend servers.

Implementation:

1. Launched three EC2 instances: **Server-1**, **Server-2**, and **HA-Proxy-Server**.
2. Installed and configured **Apache HTTP Server** on Server-1.
3. Installed and configured **Nginx** on Server-2.
4. Added the required server IP addresses in `/etc/hosts`.
5. Installed HAProxy using `yum install haproxy -y`.

- | ☐ | Name  | Instance ID | Instance state  | Instance type  | Status check | Availability Zone |
|--------------------------|--|---------------------|---|---|---|-------------------|
| ☐ | HA-Proxy-Server | i-009de607a921f5ed2 |  Running   | t3.micro |  3/3 checks passed | us-east-1c |
| ☐ | server-2 | i-025996a704e57dfd9 |  Running   | t3.micro |  3/3 checks passed | us-east-1c |
| ☐ | server-1 | i-03b7bea83cf53abdd |  Running   | t3.micro |  3/3 checks passed | us-east-1c |

```
91738@IDEAPAD MINGW64 ~/Downloads
$ ssh -i linux-testing.pem ec2-user@54.162.170.19
The authenticity of host '54.162.170.19 (54.162.170.19)' can't be established.
ED25519 key fingerprint is SHA256:hsjZrFKskK/+N3WGqqQ29taDsE68ztXzdA6TQiKiPOnA.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '54.162.170.19' (ED25519) to the list of known hosts.
```

The terminal window shows the progress of an SSH connection. On the left side, there is a graphical representation of the network path between the local machine and the remote server. It consists of several connected nodes represented by small icons like triangles and circles, forming a branching tree-like structure. To the right of this diagram, the following text appears:

```
##### Amazon Linux 2023
##### |
##### |
### #/ https://aws.amazon.com/linux/amazon-linux-2023
V~' _-_->
~~~~
~~~.-.._
    / \   ~--
   /m\  .-./
```

```
[ec2-user@ip-172-31-21-46 ~]$ sudo - 
sudo: -: command not found
[ec2-user@ip-172-31-21-46 ~]$ sudo su -
```

```

[root@ip-172-31-21-46 ~]# yum update -y
amazon linux 2023 kernel livepatch repository
Dependencies resolved.
Nothing to do.
complete!
[root@ip-172-31-21-46 ~]# yum install nginx -y
last metadata expiration check: 0:01:20 ago on Thu Aug 13 06:56:34 2026.
Dependencies resolved.

```

Package	Architecture	Version	Repository	Size
Installing:				
nginx	x86_64	1:1.30.4-1.amzn2023.0.1	amazonlinux	34 k
Installing dependencies:				
generic-logos-httpd	noarch	18.0.0-12.amzn2023.0.3	amazonlinux	19 k
querrfcoe-15-tlbs	x86_64	2.9.1-1.amzn2023.0.3	amazonlinux	308 k
libunwind	x86_64	1.4.0-5.amzn2023.0.3	amazonlinux	66 k
nginx-core	x86_64	1:1.30.4-1.amzn2023.0.1	amazonlinux	711 k
nginx-filesystem	noarch	1:1.30.4-1.amzn2023.0.1	amazonlinux	10 k
nginx-mimetypes	noarch	2.1.49-3.amzn2023.0.3	amazonlinux	21 k

```

Transaction Summary

```

```
[root@ip-172-31-21-46 ~]# vi /etc/hosts
[root@ip-172-31-21-46 ~]# cat /etc/hosts
127.0.0.1      localhost localhost.localdomain localhost4 localhost4.localdomain4
::1           localhost6 localhost6.localdomain6
13.221.133.50 load_balancer
[root@ip-172-31-21-46 ~]#
```

```
[root@ip-172-31-21-46 ~]# ping load_balancer -c 4
PING load_balancer (13.221.133.50) 56(84) bytes of data.

--- load_balancer ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3125ms

[root@ip-172-31-21-46 ~]# |
```


haproxy

```
#  
##### Amazon Linux 2023  
~\####/  
~~\###|  
~~\#/  
~~~~V~' -> https://aws.amazon.com/linux/amazon-linux-2023  
~~~~.  
~~~-./.  
_m/'/_/_/_/_
```

[ec2-user@ip-172-31-18-17 ~]\$ |

```

root@ip-172-31-18-17 ~]# yum install haproxy -y
Last metadata expiration check: 0:00:44 ago on Thu Aug 13 07:05:01 2026.
Dependencies resolved.

```

Package	Architecture	Version	Repository	Size
Installing: haproxy	x86_64	3.0.23-2.amzn2023.0.1	amazonlinux	2.7 M

```

Transaction Summary
Install 1 Package

Total download size: 2.7 M
Installed size: 9.5 M
Downloading Packages:
haproxy-3.0.23-2.amzn2023.0.1.x86_64.rpm

```

	19 MB/s 2.7 MB	00:00
Total	14 MB/s 2.7 MB	00:00

```

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing      : haproxy-3.0.23-2.amzn2023.0.1.x86_64
Running scriptlet: haproxy-3.0.23-2.amzn2023.0.1.x86_64
Installing     : haproxy-3.0.23-2.amzn2023.0.1.x86_64
Running scriptlet: haproxy-3.0.23-2.amzn2023.0.1.x86_64
Verifying      : haproxy-3.0.23-2.amzn2023.0.1.x86_64

```

Preparing	1/1
Running scriptlet	1/1
Installing	1/1
Running scriptlet	1/1
Verifying	1/1

```

Installed:
haproxy-3.0.23-2.amzn2023.0.1.x86_64

Complete!

```

```
[root@ip-172-31-18-17 ~]# vi /etc/hosts
[root@ip-172-31-18-17 ~]# cat /etc/hosts
127.0.0.1        localhost localhost.localdomain localhost4 localhost4.localdomain4
::1             localhost6 localhost6.localdomain6
54.162.170.19   server-1
54.226.120.145  server-2
[root@ip-172-31-18-17 ~]#
```

```

[root@ip-172-31-18-17 ~]# vi /etc/haproxy/haproxy.cfg
[root@ip-172-31-18-17 ~]# cat /etc/haproxy/haproxy.cfg
#-----
# Example configuration for a possible web application. See the
# full configuration options online.
#
# https://www.haproxy.org/download/1.8/doc/configuration.txt
#-----
#
# Global settings
#-----
global
# to have these messages end up in /var/log/haproxy.log you will
# need to:
#
# 1) configure syslog to accept network log events. This is done
# by adding the '-r' option to the SYSLOGD_OPTIONS in
# /etc/sysconfig/syslog
#
# 2) configure local2 events to go to the /var/log/haproxy.log
# file. A line like the following can be added to
# /etc/sysconfig/syslog
#
# local2.*                               /var/log/haproxy.log
#

```

```

# main frontend which proxys to the backends
#-----
frontend main
bind *:5000
bind *:80
acl url_static      path_beg       -i /static /images /javascript /stylesheets
acl url_static      path_end       -i .jpg .gif .png .css .js

    use_backend static      if url_static
    default_backend         app

#-----
# static backend for serving up images, stylesheets and such
#-----
backend static
balance roundrobin
server      static 127.0.0.1:4331 check

#-----
# round robin balancing between the various backends
#-----
backend app
balance roundrobin
server app1 127.0.0.1:5001 check
server app2 127.0.0.1:5002 check
server app3 127.0.0.1:5003 check
server app4 127.0.0.1:5004 check
server app5 54.162.170.19:80 check
server app6 54.226.120.145:80 check
[root@ip-172-31-18-17 ~]#

```

```

[root@ip-172-31-18-17 ~]# systemctl restart haproxy
[root@ip-172-31-18-17 ~]# systemctl status haproxy
● haproxy.service - HAProxy Load Balancer
   Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-08-13 07:24:17 UTC; 54s ago
     Process: 27290 ExecStartPre=/usr/sbin/haproxy -f $CONFIG -f $CFGDIR -c -q $OPTIONS (code=exited, status=0/SUCCESS)
    Main PID: 27292 (haproxy)
      Status: "Ready."
        Tasks: 3 (limit: 1059)
       Memory: 10.0M
          CPU: 130ms
        CGroup: /system.slice/haproxy.service
                └─27292 /usr/sbin/haproxy -ws -f /etc/haproxy/haproxy.cfg -f /etc/haproxy/conf.d
                  └─27294 /usr/sbin/haproxy -ws -f /etc/haproxy/haproxy.cfg -f /etc/haproxy/conf.d

Aug 13 07:24:17 ip-172-31-18-17.ec2.internal haproxy[27292]: [ALERT] (27292) : config : par
Aug 13 07:24:17 ip-172-31-18-17.ec2.internal haproxy[27292]: [NOTICE] (27292) : New worker (
Aug 13 07:24:17 ip-172-31-18-17.ec2.internal haproxy[27292]: [NOTICE] (27292) : Loading succ
Aug 13 07:24:17 ip-172-31-18-17.ec2.internal haproxy[27294]: [WARNING] (27294) : Server stati
Aug 13 07:24:17 ip-172-31-18-17.ec2.internal haproxy[27294]: [ALERT] (27294) : backend 'sta
Aug 13 07:24:17 ip-172-31-18-17.ec2.internal systemd[1]: Started haproxy.service - HAProxy Loa

```

Conclusion:

Successfully configured HAProxy to distribute web traffic between Server-1 and Server-2.